

Cognito and WAF

Authentication · JWT · Social Login · WAF Rules · Rate Limiting · ShopFlow Security

AWS Tutorial · ShopFlow

17-01

Authentication and Authorization for ShopFlow

■ Authentication vs Authorization

Authentication (AuthN) answers "Who are you?" — verifying identity via username/password, Google login, or biometrics. Authorization (AuthZ) answers "What can you do?" — once identity is confirmed, what actions are permitted? Amazon Cognito handles authentication for ShopFlow. IAM policies and application-level role checks handle authorization.

Example: ShopFlow customer flow: (1) Customer clicks "Sign In" (2) Cognito verifies email+password (AuthN) (3) Cognito issues a JWT token (4) Customer's browser sends JWT with every API request (5) ALB or Spring Boot verifies the JWT and extracts userId and email (6) Application checks if userId can access the requested order (AuthZ — your orders, not someone else's).

Security Layer	Handles	AWS Service	ShopFlow Example
Edge protection	Block bad traffic before it reaches the app	WAF (Web Application Firewall)	Block SQL injection in /products/search?q= ; ra
Network layer	VPC security groups; private subnets for Data S	VPC, Security Groups	ECS tasks in private subnets; RDS only reacha
Authentication	Verify user identity; issue tokens	Amazon Cognito User Pool	Customer login with email+password; Google/F
Token verification	Validate JWT on every API request	ALB + Cognito integration; Spring Boot Security	ALB rejects expired JWT; Spring Security
Authorization	What can this user do?	Application code + IAM	Customer can only view their own orders; admin
Data encryption	Protect data at rest and in transit	KMS + ACM (TLS)	Aurora encrypted with KMS; HTTPS for all traff

17-02

Amazon Cognito User Pool — Managed Authentication

A Cognito User Pool is a managed user directory. Instead of building your own login system (bcrypt passwords, session management, MFA, account logout), Cognito provides all of this out of the box for ShopFlow.

Feature	Cognito Provides	Without Cognito (DIY)
Password storage	Bcrypt hashed; never stored in plaintext; separate password database	Build application codebase; risk of plaintext pass
Email verification	Send verification email on signup; block unverified accounts	Build out own SES email flow; handle verification token ex
MFA	SMS MFA or TOTP (Google Authenticator) with one-time code generation/validation	Build TOTP generation/validation; manage SMS provider in
Social login (Google/Facebook)	Add identity provider in console; Cognito handles OAuth 2.0 client for each provider	Build OAuth 2.0 client for each provider; handle token exch
Account logout	Automatic after N failed attempts; configurable	Build your own failed-attempt counter and logout logic
Password policy	Min length, complexity, history — configurable	Implement and enforce your own password rules
JWT tokens	Issues id_token, access_token, refresh_token on login	Build your own token generation, signing, and validation

17-03

Cognito Console — Creating a User Pool for ShopFlow

Follow these steps to create the ShopFlow customer User Pool in the Cognito Console.

■ Create ShopFlow Customer User Pool

1. AWS Console -> Cognito -> User pools -> Create user pool
2. Provider types: Cognito user pool (built-in) + Google (social login) -> Next
3. Sign-in options: Email (customers sign in with email address) -> Next
4. Password policy: Cognito defaults (8 chars, uppercase, number, symbol) -> Keep defaults
5. MFA: Optional MFA (SMS or authenticator app) -> No MFA for basic setup (enable later)
6. Self-service account recovery: Enable (email recovery) -> Next
7. Self-registration: Allow users to sign themselves up (e-commerce: yes) -> Next
8. Email: Cognito sends verification emails (free tier: 50/day; production: configure SES)
9. User pool name: shopflow-customer-pool -> Next
10. App client name: shopflow-web-app | Client type: Public (no secret; React SPA) -> Next
11. Review and create user pool

■ aws > Cognito > User pools > shopflow-customer-pool

User Pool: shopflow-customer-pool

■ User pool overview

- User pool ID: us-east-1_AbCdEfGhI | Region: us-east-1
- Total users: 1,247,832 | Enabled users: 1,241,109

■ Sign-in options

- Username: Email address
- Social identity providers: Google (OAuth 2.0)

■ App clients

- shopflow-web-app | Client ID: 1a2b3c4d5e | Type: Public (no secret)
- shopflow-mobile-app | Client ID: 6f7g8h9i0j | Type: Public (no secret)

■ Advanced security

- Compromised credentials: Enabled (blocks known breached passwords)
- Adaptive authentication: Enabled (extra MFA challenge on suspicious login)

17-04

Cognito CDK — Complete User Pool Configuration

ShopFlow defines the Cognito User Pool in CDK so it can be version-controlled, peer-reviewed, and deployed consistently across environments.

```
// CDK: ShopFlow Cognito User Pool
UserPool userPool = UserPool.Builder.create(this,
"CustomerPool") .userPoolName("shopflow-customer-pool") // Sign-in: email only (not username)
.signInAliases(SignInAliases.builder().email(true).build())
.autoVerify(AutoVerifiedAttrs.builder().email(true).build()) // Standard attributes (collected
at signup) .standardAttributes(StandardAttributes.builder()
.email(StandardAttribute.builder().required(true).mutable(false).build())
```

```

.givenName(StandardAttribute.builder().required(true).mutable(true).build())
.familyName(StandardAttribute.builder().required(true).mutable(true).build()) .build() //
Password policy .passwordPolicy(PasswordPolicy.builder() .minLength(8).requireUppercase(true)
.requireDigits(true).requireSymbols(true) .tempPasswordValidity(Duration.days(7)) .build()) //
Account recovery: email only (not SMS; avoid phone number requirement)
.accountRecovery(AccountRecovery.EMAIL_ONLY) // Self-registration allowed (e-commerce:
customers sign themselves up) .selfSignUpEnabled(true) // MFA: optional (customers can enable
it; not required) .mfa(Mfa.OPTIONAL)
.mfaSecondFactor(MfaSecondFactor.builder().sms(true).otp(true).build()) // Advanced security:
block known breached passwords .advancedSecurityMode(AdvancedSecurityMode.ENFORCED) // Deletion
protection (never accidentally delete 1M user accounts) .deletionProtection(true) .build(); //
App client for React/React Native SPA (no client secret) UserPoolClient webClient =
userPool.addClient("WebClient", UserPoolClientOptions.builder()
.userPoolClientName("shopflow-web-app") .authFlows(AuthFlow.builder() .userSrp(true) // SRP:
password never sent in plaintext .userPassword(false) // disable basic auth (less secure)
.build()) .oAuth(OAuthSettings.builder()
.flows(OAuthFlows.builder().authorizationCodeGrant(true).build())
.scopes(List.of(OAuthScope.EMAIL, OAuthScope.OPENID, OAuthScope.PROFILE))
.callbackUrls(List.of("https://shopflow.com/auth/callback"))
.logoutUrls(List.of("https://shopflow.com/logout")) .build())
.accessTokenValidity(Duration.minutes(60)) .idTokenValidity(Duration.minutes(60))
.refreshTokenValidity(Duration.days(30)) .build());

```

17-05

JWT Tokens — What Cognito Returns After Login

After a successful login, Cognito returns three JWT tokens. The React frontend stores these in memory (not localStorage) and sends the Access Token with every API request.

Token	Contents	Validity	ShopFlow Use
ID Token	User claims: sub (userId), email, given_name, family_name, cognito:groups (Spring Boot extracts userId and email from this)	60 minutes	
Access Token	OAuth scopes, username, jti (token ID), token_use=access	60 minutes	Sent in Authorization: Bearer header with token
Refresh Token	Opaque (not a JWT); used to get new Access/ID tokens via refresh-token	90 days	React frontend exchanges Refresh Token for new Access/ID tokens

```

// JWT payload of ShopFlow ID Token (decoded – base64) { "sub":
"a1b2c3d4-e5f6-7890-abcd-ef1234567890", // Cognito user ID (UUID) "email": "john@example.com",
"given_name": "John", "family_name": "Doe", "email_verified": true, "cognito:groups":
["customers"], // group membership (used for AuthZ) "cognito:username": "john@example.com",
"iss": "https://cognito-idp.us-east-1.amazonaws.com/us-east-1_AbCdEfGhI", "aud": "1a2b3c4d5e",
// app client ID "iat": 1718640000, // issued at (Unix timestamp) "exp": 1718643600, // expires
at (1 hour later) "token_use": "id" } // Spring Boot: extract userId from JWT @Component public
class JwtUserExtractor { public String getUserId(Authentication auth) { JwtAuthenticationToken
jwt = (JwtAuthenticationToken) auth; return jwt.getToken().getClaimAsString("sub"); // Cognito
UUID } }

```

17-06

ALB + Cognito — Authenticate at the Load Balancer

ShopFlow configures the ALB to verify Cognito JWTs on every request before forwarding to ECS. Unauthenticated requests receive a 401 Unauthorized response from the ALB — the ECS task never sees them.

```

// CDK: ALB listener rule with Cognito authentication UserPoolDomain userPoolDomain =
UserPoolDomain.Builder.create(this, "PoolDomain") .userPool(userPool)
.cognitoDomain(CognitoDomainOptions.builder() .domainPrefix("shopflow-auth") // ->
shopflow-auth.auth.us-east-1.amazoncognito.com .build()) .build(); // ALB HTTPS listener:
authenticate all /api/* requests via Cognito httpsListener.addAction("AuthenticatedApi",
AddApplicationActionProps.builder() .priority(10)
.conditions(List.of(ListenerCondition.pathPatterns(List.of("/api/*"))))
.action(ListenerAction.authenticateCognito( AuthenticateCognitoOptions.builder()
.userPool(userPool) .userPoolClient(webClient) .userPoolDomain(userPoolDomain)
.onUnauthenticatedRequest(UnauthenticatedAction.DENY) // return 401 // ALB adds
X-Amzn-Oidc-Data header (JWT user info) to forwarded request // Spring Boot reads this header –
no JWT parsing library needed .build(), ListenerAction.forward(List.of(apiTargetGroup))))
.build()); // Spring Boot: read authenticated user from ALB header (X-Amzn-Oidc-Data)
@RestController public class OrderController { @GetMapping("/api/orders") public List[Order]

```

```
getMyOrders( @RequestHeader("X-Amzn-Oidc-Data") String oidcDataJwt) { // ALB already validated the token; just decode the header String userId = JwtDecoder.decodeHeader(oidcDataJwt).getClaim("sub"); return orderService.findByUserId(userId); } }
```

17-07

Social Login — Google Sign-In for ShopFlow

ShopFlow allows customers to sign in with their Google account instead of creating a separate password. Cognito handles the OAuth 2.0 flow with Google — the application just uses Cognito as always.

■ Add Google Sign-In to Cognito User Pool

1. Google Cloud Console: APIs & Services -> Credentials -> Create OAuth 2.0 Client ID
2. Application type: Web application | Authorized redirect URI: <https://shopflow-auth.auth.us-east-1.amazoncognito.com/oauth2/idpresponse>
3. Copy Client ID and Client Secret from Google
4. Cognito -> shopflow-customer-pool -> Sign-in experience -> Add identity provider
5. Provider: Google | Google App ID: (paste Client ID) | App secret: (paste Client Secret)
6. Authorized scopes: profile email openid
7. Map attributes: email -> email, given_name -> given_name, sub -> username
8. App client -> shopflow-web-app -> Edit hosted UI: enable Google as identity provider
9. Callback URL: <https://shopflow.com/auth/callback> (must match Google OAuth config)
10. ShopFlow UI now shows "Continue with Google" button on login page

Google Sign-In Flow Step	Who Does What
Customer clicks "Continue with Google"	React redirects to Cognito Hosted UI URL (no Google code in React)
Cognito Hosted UI shows Google button	Cognito redirects to Google's OAuth server with client_id and redirect_uri
Customer sees Google permission screen	Google authenticates user; asks permission to share email/name
Google redirects to Cognito callback	Google sends authorization code to Cognito's /oauth2/idpresponse endpoint
Cognito exchanges code with Google	Cognito calls Google API with code; gets user profile (email, name)
Cognito creates/finds user in User Pool	If first Google login: creates new Cognito user. Subsequent logins: finds existing user.
Cognito redirects to shopflow.com/auth/callback	Authorization code sent to React frontend
React exchanges code for tokens	React calls Cognito /oauth2/token endpoint; gets id_token, access_token, refresh_token
Normal ShopFlow login experience	JWT used for all subsequent API calls — no difference from email/password login

17-08

AWS WAF — Web Application Firewall

AWS WAF inspects HTTP requests before they reach the ALB. Rules can block, allow, or count requests based on IP address, request headers, URI path, query strings, or request body content. ShopFlow uses WAF as the first line of defense against attacks.

■ WAF WebACL

A WebACL (Web Access Control List) is a collection of WAF rules. Each rule has an action: Block (return 403), Allow (forward request), or Count (log for analysis). Rules are evaluated in priority order (lowest number first). The WebACL has a default action (Allow or Block) applied if no rule matches.

Example: ShopFlow WAF attack scenario: a bot sends 10,000 requests to `/products/search?q=' OR 1=1--` (SQL injection). WAF Rule 1 (AWS Managed SQL injection) catches it and blocks the request at the CDN edge before it reaches ECS. The application never sees the malicious request. CloudWatch WAF metrics show the blocked requests.

17-09

WAF Rules — ShopFlow WebACL Configuration

ShopFlow's WAF WebACL has six rules. AWS Managed Rule Groups cover common attacks automatically; custom rules handle ShopFlow-specific requirements.

Priority	Rule Name	Type	What It Does	Action
1	AWSManagedRulesCommonRuleSet	AWS Managed	OWASP Top 10: SQL injection, XSS, path traversal, SSRF, etc.	Block matching requests
2	AWSManagedRulesKnownBadInputsRuleSet	AWS Managed	Blocks requests with known malicious patterns: Log4Shell, Spring4Shell, etc.	Block matching requests
3	AWSManagedRulesAmazonIpReputationRuleSet	AWS Managed	Blocks IP addresses from AWS threat intelligence: IP addresses associated with malicious activity.	Block matching requests
4	ShopFlowRateLimitRule	Custom	Rate limit: > 1000 requests per 5 minutes per IP address.	Block. Presents rate force and
5	ShopFlowCheckoutRateLimit	Custom	Rate limit: > 20 POST /orders per 5 minutes per IP.	Block. Presents automated checkout
6	AWSManagedRulesBotControlRuleSet	AWS Managed (paid add-on)	Detects and block automated bot traffic using behavioral analysis.	Block. Allows known good

17-10

WAF CDK Configuration

ShopFlow defines the WAF WebACL in CDK. WAF is a regional resource (for ALB protection) — not global (global is for CloudFront only).

```
// CDK: ShopFlow WAF WebACL (REGIONAL scope for ALB) CfnWebACL webAcl =
CfnWebACL.Builder.create(this, "ShopFlowWAF") .name("shopflow-waf-prod") .scope("REGIONAL") //
REGIONAL for ALB; CLOUDFRONT for CloudFront (different region)
.defaultAction(CfnWebACL.DefaultActionProperty.builder() .allow(Map.of()) // allow by default;
rules block specific patterns .build())
.visibilityConfig(CfnWebACL.VisibilityConfigProperty.builder() .cloudWatchMetricsEnabled(true)
.metricName("ShopFlowWAF") .sampledRequestsEnabled(true) // sample blocked requests for
analysis .build()) .rules(List.of( // AWS Managed: OWASP Top 10
managedRule("AWSManagedRulesCommonRuleSet", 1), // AWS Managed: Known bad inputs (Log4Shell,
etc.) managedRule("AWSManagedRulesKnownBadInputsRuleSet", 2), // Custom: rate limit per IP
CfnWebACL.RuleProperty.builder() .name("RateLimit1000") .priority(4)
.action(CfnWebACL.RuleActionProperty.builder() .block(Map.of()).build())
.statement(CfnWebACL.StatementProperty.builder()
.rateBasedStatement(CfnWebACL.RateBasedStatementProperty.builder() .limit(1000)
.aggregateKeyType("IP") .evaluationWindowSec(300) // 1000 req per 5 minutes .build()) .build())
.visibilityConfig(visibilityConfig("RateLimit1000")) .build() )) .build(); // Associate WAF
WebACL with ALB CfnWebACLAssociation.Builder.create(this, "WAFAssociation")
.resourceArn(alb.getLoadBalancerArn()) .webAclArn(webAcl.getAttrArn()) .build();
```

17-11

WAF Console — Viewing Blocked Requests

The WAF Console lets you see which requests were blocked, by which rule, and from which IP. This is essential for tuning WAF rules to avoid false positives (blocking legitimate users).

■ View WAF Blocked Requests

1. AWS Console -> WAF & Shield -> Web ACLs -> shopflow-waf-prod
2. Click "Overview" tab -> see Requests chart and top rules by block count
3. Click "Traffic overview" -> set time range: Last 24 hours
4. Chart shows: Allowed requests (blue) vs Blocked requests (red)
5. Click "Sampled requests" tab -> see specific blocked requests
6. Each row shows: Timestamp, Source IP, HTTP method, URI, Matched Rule, Response code
7. Click a row -> expands to show full request headers and body (for debugging false positives)
8. Click "Rules" tab -> per-rule metrics: how many requests matched each rule
9. Set a rule to "Count" mode (instead of Block) to analyze before enforcing

■ aws > WAF > Web ACLs > shopflow-waf-prod > Sampled Requests

WAF Sampled Requests — Last 1 Hour

■ WAF Traffic (Last 1 Hour)

■ Total requests: 2,847,332 | Allowed: 2,841,109 | Blocked: 6,223 (0.22%)

■ Blocked Request Samples

- IP: 185.220.101.45 | POST /orders | Rule: RateLimit-Checkout | 403 | 14:23:01
- IP: 91.108.4.182 | GET /products/search?q=UNION+SELECT | Rule: CommonRuleSet | 403 | 14:22:44
- IP: 45.155.205.89 | GET / | Rule: AmazonIpReputationList | 403 | 14:22:31
- IP: 192.168.1.100 | GET /admin | Rule: CommonRuleSet (false positive?) | 403 | 14:22:01

■ Top Blocked Rules

- AWSManagedRulesCommonRuleSet: 3,891 blocked (62.5%)
- AmazonIpReputationList: 1,547 blocked (24.9%)
- RateLimit1000: 785 blocked (12.6%)

17-12

Cognito Groups — Role-Based Access Control

Cognito Groups assign users to roles (customers, sellers, admins). The group membership is included in the JWT token. Spring Boot uses this for authorization — determining what each user is allowed to do.

```
// CDK: Create Cognito Groups for ShopFlow // "customers" group: default for all new signups
UserPoolGroup customersGroup = UserPoolGroup.Builder.create(this, "CustomersGroup")
    .userPool(userPool) .groupName("customers") .description("All ShopFlow customers – can browse,
add to cart, place orders") .build(); // "sellers" group: merchants who list products on
ShopFlow marketplace UserPoolGroup sellersGroup = UserPoolGroup.Builder.create(this,
"SellersGroup") .userPool(userPool) .groupName("sellers") .description("Marketplace sellers –
can create/edit products; view their own orders") .build(); // "admins" group: ShopFlow
operations team UserPoolGroup adminsGroup = UserPoolGroup.Builder.create(this, "AdminsGroup")
.userPool(userPool) .groupName("admins") .description("ShopFlow admin team – can view all
orders; manage users; access analytics") .build(); // JWT token for an admin user includes:
"cognito:groups": ["admins"] // Spring Boot Security: role-based access via group membership
from JWT @Configuration @EnableWebSecurity public class SecurityConfig { @Bean public
```

```
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth .requestMatchers("/api/products/**").permitAll() //
    public product browsing
    .requestMatchers("/api/cart/**").hasAnyRole("customers","sellers","admins")
    .requestMatchers("/api/orders/**").hasAnyRole("customers","sellers","admins")
    .requestMatchers("/api/admin/**").hasRole("admins") // admin panel: admins only
    .requestMatchers("/api/seller/**").hasAnyRole("sellers","admins") .anyRequest().authenticated()
    ); return http.build(); } }
```

Complete view of ShopFlow's security layers from the internet to the database.

ShopFlow Security Architecture

Edge Layer — Internet-facing

AWS WAF WebACL — shopflow-waf-prod

CloudFront — DDoS protection (AWS Shield Standard)

Authentication Layer

Cognito User Pool — shopflow-customer-pool

ALB Cognito Integration

Application Layer — ECS Fargate Private Subnets

Spring Boot Security — RBAC from JWT groups

Data Layer — Isolated Subnets

Aurora PostgreSQL — encrypted KMS; SG allows ECS only (port 5432)

DynamoDB — encrypted; VPC endpoint; no internet access

ElastiCache Redis — AUTH token; SG allows ECS only (port 6379)

Secrets Management

SSM Parameter Store SecureString — DB passwords, API keys

Rotation: Aurora password rotates every 30 days (Secrets Manager Lambda)

In this lab you will create a Cognito User Pool, add a test user, and set up a basic WAF WebACL with SQL injection protection. Time estimate: 40 minutes.

■ Lab: Cognito User Pool + WAF

1. Cognito -> Create user pool -> Email sign-in -> defaults for everything else
2. User pool name: shopflow-lab-pool | App client: shopflow-lab-client (public, no secret)
3. Create user pool -> note the Pool ID (us-east-1_XXXXX)
4. Create a test user: Users -> Create user -> email: test@example.com -> temp password
5. Test sign-in: Cognito Hosted UI -> open the app client URL in browser
6. Sign in with test credentials -> Cognito issues JWT tokens (visible in URL after redirect)
7. Decode the JWT at jwt.io -> verify claims: sub, email, cognito:groups
8. WAF -> Create web ACL -> Region: us-east-1 | Name: shopflow-lab-waf
9. Add managed rule group: AWSManagedRulesCommonRuleSet
10. Add custom rule: Rate limit 100 req/IP/5min (lower for lab testing)
11. Associate with your ALB (or skip association if no ALB in lab)
12. Test WAF: send SQL injection request -> verify 403 response
13. Cleanup: delete WAF WebACL, Cognito user pool, app client

Cognito Lambda Triggers let you run custom code at specific points in the authentication flow. ShopFlow uses triggers to add userId to the JWT, migrate legacy users, and send custom welcome emails.

Trigger	When It Fires	ShopFlow Use
Pre-signup	Before a new account is created	Validate email domain (block disposable email addresses like mailinator.com)
Post-confirmation	After user verifies email	Create user record in Aurora (profile table); send welcome email via SES;
Pre-token generation	Before Cognito issues JWT tokens	Add custom claims to JWT: shopflowUserId (from Aurora users table), sub
Pre-authentication	Before verifying credentials	Check if account is banned in ShopFlow database; return error if banned
Migrate user	When user logs in but not found in User Pool	Lookup email in legacy system; if found, create Cognito account and imp

```
// Lambda trigger: pre-token generation (add ShopFlow userId to JWT) public class
PreTokenTrigger { private final JdbcTemplate jdbc; public Map[String, Object] handleRequest(
Map[String, Object] event, Context ctx) { String cognitoUserId = (String)
event.get("userName"); String email = (String) ((Map)
event.get("request")).get("userAttributes"); // note: userAttributes is a Map; get email from
it Map[String, Object] userAttributes = (Map[String, Object]) ((Map)
event.get("request")).get("userAttributes"); String emailVal = (String)
userAttributes.get("email"); // Look up internal ShopFlow userId from Aurora String
shopflowUserId = jdbc.queryForObject( "SELECT id FROM users WHERE email = ?", String.class,
emailVal); // Add custom claim to the JWT token Map[String, Object] response = (Map[String,
Object]) event.get("response"); Map[String, Object] claimsToAdd = new HashMap();
claimsToAdd.put("shopflowUserId", shopflowUserId); Map[String, Object] claimsOverride = new
HashMap(); claimsOverride.put("claimsToAddOrOverride", claimsToAdd);
response.put("claimsOverrideDetails", claimsOverride); return event; } } // JWT now includes:
"shopflowUserId": "USR-12345" as a custom claim // Spring Boot reads it:
jwt.getClaim("custom:shopflowUserId")
```

WAF can log every request (allowed and blocked) to an S3 bucket or Kinesis Firehose. ShopFlow enables WAF logging and uses Athena to analyze attack patterns and tune rules.

```
// CDK: Enable WAF logging to Kinesis Firehose -> S3
CfnLoggingConfiguration wafLogs =
CfnLoggingConfiguration.Builder.create(this, "WAFLogs")
    .resourceArn(webAcl.getAttrArn())
    .logDestinationConfigs(List.of( // Kinesis Firehose -> S3 (WAF requires Firehose; cannot log
    directly to S3) firehoseDeliveryStream.getAttrArn())) // Log all requests (not just blocked) to
    understand traffic patterns // Filter to only log blocked requests to reduce cost:
    .loggingFilter(CfnLoggingConfiguration.LoggingFilterProperty.builder()
        .defaultBehavior("DROP")
        // do not log allowed requests (too many)
        .filters(List.of(CfnLoggingConfiguration.FilterProperty.builder()
            .behavior("KEEP")
            .requirement("MEETS_ANY")
            .conditions(List.of(CfnLoggingConfiguration.ConditionProperty.builder()
                .actionCondition(CfnLoggingConfiguration.ActionConditionProperty.builder()
                    .action("BLOCK") //
                    only log blocked requests .build()) .build())) .build()) .build(); -- Athena query:
    Top attacking IPs over last 7 days
    SELECT httpRequest.clientIp, COUNT(*) as blockCount,
    ARRAY_AGG(DISTINCT terminatingRuleId) as rulesTriggered
    FROM waf_logs WHERE action = "BLOCK"
    AND from_unixtime(timestamp/1000) > current_date - interval "7" day
    GROUP BY
    httpRequest.clientIp ORDER BY blockCount DESC LIMIT 20;
```

Practice	Details
Never store JWTs in localStorage	localStorage is accessible to JavaScript and vulnerable to XSS attacks. Store access tokens in memory.
Short access token validity	ShopFlow sets access tokens to 60 minutes (default is 1 hour). A stolen token is only valid for the remainder of its validity.
Enable Cognito Advanced Security	Advanced Security adds: compromised credential checks (blocks known breached passwords), adaptive risk analysis, and more.
WAF in Count mode before Block mode	When adding a new WAF rule, set it to "Count" first for 1 week. Review sampled requests to ensure rule is correct.
Regularly review WAF false positives	Check WAF sampled blocked requests weekly. If legitimate user traffic is blocked (e.g., a product screenshot), remove the rule.
Use Cognito groups for coarse-grained authorization	Cognito groups (customers/sellers/admins) handle role-based access. Fine-grained authorization (e.g., product categories) is handled by rules.
Rotate secrets regularly	Aurora passwords rotate every 30 days via Secrets Manager. Cognito app client secrets (if using cognito-js) rotate every 90 days.

Service	Pricing	ShopFlow Monthly Volume	Monthly Cost
Cognito — MAU	Free for first 50,000 MAU; \$0.0055/MAU after	1M MAU -> 950K paid MAUs	\$5,225/month
Cognito — Advanced Security	\$0.05 per MAU when enabled	1M MAU	\$50,000/month (consider: disable for inactive users)
Cognito — SAML/OIDC Federation	Free for first 50/month; \$0.015/MAU after	0 (ShopFlow uses direct Cognito + Google OAuth, not enterprise SAML)	\$0
WAF WebACL	\$5/month per WebACL	1 WebACL	\$5
WAF — Rule groups	\$1/month per rule group	3 AWS managed + 2 custom = 5 rule groups	\$5
WAF — Requests	\$0.60 per 1M requests	300M requests/month (all traffic inspected)	\$180
WAF — Bot Control (adds \$10/M WebACL/month + \$1/M requests)		If enabled: \$10 + \$300	\$310 (optional)

INFO Cognito MAU Cost at Scale

Cognito Advanced Security at \$0.05/MAU x 1M MAU = \$50,000/month is significant. ShopFlow disables Advanced Security for inactive users (users with no login in 90 days are moved to a separate pool without Advanced Security). For 200K active MAU the cost is \$10,000/month — more manageable. Alternatively, consider Cognito Lite (coming in 2025) or self-hosted Keycloak for very high MAU workloads.

Problem / Question	Diagnosis / Answer
User gets 401 on API calls but token is valid	Check if token is expired: JWT exp claim is a Unix timestamp. Check ALB Cognito integration is valid
WAF is blocking legitimate users (false positive)	WAF -> Sampled Requests; find the blocked request; check which rule matched. Set that rule to 'Allow'
Google sign-in works but user gets wrong email address	Check Cognito attribute mapping for Google: ensure Cognito sub is mapped to Google sub (not email)
Refresh token expired — user gets logged out	This is safe behavior. The ShopFlow React app uses Amplify's auto-refresh: it exchanges refresh token for new one before it expires (2 days)
How to add a B2B admin user without public self-registration	Create self-registration Cognito User Pool for internal/admin users with self-registration disabled. Add them manually

A Cognito Identity Pool (Federated Identity) gives authenticated users temporary AWS credentials (via STS). ShopFlow uses this for the seller mobile app: sellers can upload product photos directly to S3 without going through a server.

Aspect	Cognito User Pool	Cognito Identity Pool
What it is	User directory: stores usernames, passwords, groups, attributes	Temporary credential broker: exchanges tokens for temporary AWS credentials
Issues	JWT tokens (id_token, access_token, refresh_token)	Temporary IAM credentials (AWS Access Key ID + Secret Access Key)
Used for	Authenticating application users; API authorization	Direct AWS service access from client apps (S3 upload, DynamoDB)
Works with	Email/password, Google, Facebook, SAML, OIDC	Cognito User Pool tokens, Google, Facebook, SAML, unauthenticated
ShopFlow use	Customer authentication on web + mobile	Seller mobile app: temporary S3 credentials for product photo upload

```
// Seller mobile app: use Identity Pool to get S3 upload credentials // 1. Seller logs in via Cognito User Pool -> gets id_token // 2. Exchange id_token for S3 credentials via Identity Pool
// CDK: Identity Pool configuration
CfnIdentityPool identityPool = CfnIdentityPool.Builder.create(this, "SellerIdentityPool")
    .identityPoolName("shopflow-seller-identity-pool")
    .allowUnauthenticatedIdentities(false) // require login
    .cognitoIdentityProviders(List.of(CfnIdentityPool.CognitoIdentityProviderProperty.builder()
        .clientId(sellerUserPool.getUserPoolClientId())
        .providerName(sellerUserPool.getUserPoolProviderName())
        .build()))
    .build(); // IAM role granted to authenticated sellers (via Identity Pool)
Role sellerRole = Role.Builder.create(this, "SellerRole")
    .assumedBy(new FederatedPrincipal("cognito-identity.amazonaws.com", Map.of("StringEquals", Map.of("cognito-identity.amazonaws.com:aud", identityPool.getRef()), "sts:AssumeRoleWithWebIdentity"))
    .build(); // Only allow upload to THEIR own seller prefix in S3
// ${cognito-identity.amazonaws.com:sub} is the seller's identity ID
sellerRole.addToPolicy(PolicyStatement.Builder.create()
    .actions(List.of("s3:PutObject"))
    .resources(List.of("arn:aws:s3:::shopflow-product-images/sellers/${cognito-identity.amazonaws.com:sub}/*"))
    .build()); // React Native seller app: get S3 credentials and upload directly
// const creds = await Auth.currentCredentials(); // Amplify gets temp IAM creds
// const s3 = new S3Client({ credentials: creds, region: "us-east-1" }); // await s3.send(new PutObjectCommand({ Bucket: "shopflow-product-images", // Key: `sellers/${creds.identityId}/product-${productId}.jpg`, Body: imageFile }));
```

AWS Shield provides DDoS (Distributed Denial of Service) protection. Shield Standard is free and enabled automatically. Shield Advanced adds 24/7 DDoS response team, cost protection, and enhanced detection.

Feature	Shield Standard (free)	Shield Advanced (\$3,000/month)
Protection level	Common, volumetric DDoS attacks (Layer 3/4: network, transport)	Common, volumetric DDoS attacks (Layer 3/4/7: application-layer attacks)

Feature	Shield Standard (free)	Shield Advanced (\$3,000/month)
Visibility	No visibility into attacks; just protection	Real-time attack visibility in Shield console; attack diagnostics
DDoS Response Team	None	24/7 AWS DDoS Response Team (DRT) — escalate during att
Cost protection	None	If DDoS causes AWS costs to spike, AWS provides service cre
WAF integration	Works with WAF	Integrated WAF management during attacks; DRT applies WAF
ShopFlow decision	Currently using Standard	Plan: upgrade to Advanced before Black Friday (high revenue,

17-22

AWS Security Hub — Unified Security Findings

Security Hub aggregates security findings from WAF, GuardDuty, Inspector, Macie, IAM Access Analyzer, and Firewall Manager into a single dashboard. ShopFlow uses it to maintain a security posture score and track remediation of findings.

■ Enable and Review Security Hub

1. AWS Console -> Security Hub -> Go to Security Hub -> Enable Security Hub
2. Enable standards: AWS Foundational Security Best Practices (FSBP) + PCI DSS 3.2.1
3. Enable integrations: GuardDuty, Inspector, Macie (if used)
4. Security Hub -> Summary -> Security score appears (0-100%)
5. Click "Findings" -> filter by Severity: CRITICAL
6. Each finding shows: Resource, Issue, Remediation recommendation
7. Example critical finding: "S3 bucket shopflow-logs has public read access"
8. Click finding -> Remediation: CDK change to block public access + redeploy
9. Mark as resolved -> security score increases
10. Set up EventBridge rule: new CRITICAL finding -> SNS -> Slack alert to security team

17-23

Black Friday Security — Extra Hardening

Black Friday brings 10x traffic — and 10x attack attempts. ShopFlow increases WAF sensitivity and adds temporary rules for the high-traffic window.

Security Action	Normal Policy	Black Friday Action	Timing
WAF rate limit	1000 req/IP/5min	500 req/IP/5min (more aggressive)	Apply 48h before; revert Monday
Checkout rate limit	20 POST /orders/IP/5min	10 POST /orders/IP/5min (bot buyers)	Apply 24h before
IP reputation list	AWS managed (auto-updated)	Add manual blocklist of known credential-stuffing Add 1 week before	Check 2 weeks before
Cognito Advanced Security enabled	Enabled	Verify not hitting Cognito API limits (high login volume) limits 1 week before	Check
CloudWatch WAF alarm	5xx > 1% for 3 min	5xx > 0.5% for 1 min (more sensitive)	Apply 48h before
GuardDuty	Normal detection	Review findings daily (not weekly) during BF week	Active monitoring all week

17-24

Chapter Summary — ShopFlow Security Stack

You now understand the full security layer for ShopFlow, from blocking attacks at the edge to authenticating users to encrypting data at rest. Here is what each service provides.

Service	Core Function	ShopFlow Config
AWS WAF	Block malicious HTTP requests before they hit the application	CloudWAF managed, IP reputation, rate limiting (1000/IP/5m)
Cognito User Pool	Managed user directory: login, JWT tokens, password policies, MFA	Google OAuth; groups (customers/sellers)
Cognito Identity Pool	Exchange JWTs for temporary IAM credentials for AWS access	Select AWS app, temp S3 credentials for product photo upload
ALB + Cognito integration	Validate JWT on every API request at the load balancer layer	API endpoints require valid Cognito JWT; unauthenticated requests
Cognito Lambda Trigger	Custom logic in auth flow	Pre-token: add shopflowUserId to JWT. Post-confirmation: create
SSM Parameter Store	Secure storage for database passwords and API keys	Aurora password, Redis auth token, third-party API keys; ECS fetc
AWS Shield Standard	Free DDoS protection against network-layer attacks	Automatically enabled; upgrading to Advanced (\$3,000/month) be
Security Hub	Unified view of all security findings	PCI-DSS + FSBP standards enabled; CRITICAL findings trigger S

17-25 IAM Best Practices for ShopFlow

IAM (Identity and Access Management) governs who can do what in the AWS account. ShopFlow follows strict IAM hygiene to minimize blast radius if any credential is compromised.

IAM Best Practice	ShopFlow Implementation
Root account: lock it down	Root account has no access keys; MFA enabled; used only for billing and account recovery. All work done
Least privilege for ECS task roles	Each ECS service (product-svc, order-svc) has its own IAM task role. product-svc can read product Dyna
No long-lived IAM user access keys	Engineers use AWS SSO (IAM Identity Center) for console and CLI access. Temporary credentials, not l
Service control policies (SCPs)	AWS Organization SCP prevents any account from disabling CloudTrail, creating public S3 buckets, or c
Permission boundaries	Developer IAM roles have permission boundaries that prevent privilege escalation: developers cannot cr
IAM Access Analyzer	Monthly review of IAM Access Analyzer findings: external access to S3 buckets, cross-account access, u

17-26 Amazon GuardDuty — Threat Detection

GuardDuty continuously monitors CloudTrail, VPC Flow Logs, and DNS logs for suspicious activity using AWS threat intelligence and machine learning. ShopFlow enables it in all accounts.

GuardDuty Finding	What It Means	ShopFlow Response
UnauthorizedAccess:IAMUser/TorIP	Call IAM call was made from a TOR exit node —	Immediately investigate the access AWS ARN
CryptoCurrency:EC2/BitcoinTool.B	EC2 instance is making DNS lookups associated with the mining che possible	Identify the EC2 instance; check the possible compromise
Recon:EC2/PortProbeUnprotectedPort	external IP is probing EC2 ports — reconna	Blocked before a full PaaS WAF; review
UnauthorizedAccess:IAMUser/ConsoleLogin	Successful login from a suspicious IP (check with the device)	Identify the user; contact the account owner; if un
Exfiltration:S3/ObjectRead.Unusual	Unusual volume of S3 GetObject calls — possibl	Identify the IAM principal; revoke cr

17-27 Compliance — PCI-DSS for Payment Processing

ShopFlow processes payments (credit cards), which requires PCI-DSS compliance. AWS provides a large number of pre-certified services. ShopFlow's responsibility is in how it configures and uses them.

PCI-DSS Requirement	AWS Responsibility	ShopFlow Responsibility
Encrypt cardholder data in transit (Req 4)	ACQ certificates; TLS on ALB	Enforce HTTPS-only; no TLS 1.0/1.1; redirect HTTP to HTTPS
Encrypt cardholder data at rest (Req 3)	KMS encryption available for all services	Enable encryption on Aurora, DynamoDB, S3; ShopFlow does
Restrict access to system components (Req 7)	VPC Subnets, IAM available	ECS in private subnets; principle of least privilege for all IAM ro

PCI-DSS Requirement	AWS Responsibility	ShopFlow Responsibility
Track and monitor all access (Req 10)	CloudTrail, CloudWatch available	Enable CloudTrail in all accounts; CloudWatch Logs with 1-year retention
Regularly test security systems (Req 11)	Security Inspector, GuardDuty available	Quarterly penetration tests; monthly WAF rule review; weekly SAST

INFO Stripe for Card Data

ShopFlow does NOT store credit card numbers in Aurora or anywhere in AWS. ShopFlow uses Stripe: the customer's card number is entered directly into a Stripe-hosted form (Stripe Elements in the browser) and never sent to ShopFlow servers. Stripe returns a token (e.g., tok_visa_1234) that ShopFlow stores in DynamoDB. This dramatically reduces PCI-DSS scope — ShopFlow is a SAQ A merchant, not SAQ D.

17-28

Chapter Quick Reference

Key commands, endpoints, and configurations for Cognito and WAF.

Task	Command / Setting
Create Cognito user (CLI)	aws cognito-idp admin-create-user --user-pool-id us-east-1_XXX --username user@email.com
Add user to group (CLI)	aws cognito-idp admin-add-user-to-group --user-pool-id us-east-1_XXX --username user@email.com --group-name Developers
Decode JWT (bash)	echo "TOKEN" cut -d. -f2 base64 -d 2>/dev/null python3 -m json.tool
Cognito Hosted UI URL	https://shopflow-auth.auth.us-east-1.amazoncognito.com/login?client_id=XXX&response_type=code
Exchange auth code for tokens	POST https://shopflow-auth.auth.us-east-1.amazoncognito.com/oauth2/token (Content-Type: application/x-www-form-urlencoded)
Refresh access token	POST same /oauth2/token endpoint with grant_type=refresh_token&refresh_token=XXX&client_id=XXX
WAF: set rule to Count mode	WAF Console -> WebACL -> Rules -> Edit rule -> Action: Count (instead of Block)
WAF: view blocked requests	WAF Console -> WebACL -> Sampled requests tab
WAF: get metrics in CLI	aws wafv2 get-sampled-requests --web-acl-arn ARN --rule-metric-name RuleName --scope Region:AWS::US::US-East-1
Cognito User Pool JWKS URL	https://cognito-idp.us-east-1.amazonaws.com/us-east-1_XXX/.well-known/jwks.json (used to verify tokens)

17-29

Passkeys and Future Authentication Trends

Passwords are the weakest link in authentication — stolen via phishing, reused across sites, or brute-forced. Passkeys (FIDO2/WebAuthn) replace passwords with device-based cryptographic keys. Cognito is adding passkey support in 2025-2026.

Auth Method	Security Level	User Experience	ShopFlow Roadmap
Email + Password (current)	Low-Medium: phishable; user reuse of passwords	Familiar, everywhere; no setup	Current default; 1M existing accounts
Email + Password + MFA (optional)	High: phishing-resistant with 2FA	Extra step, SMS/MFA app	Enabled optional; 8% of users have MFA
Google/Apple Sign-In (current)	High: delegates auth to identity providers	Only pick one; Google has strong branding	Enabled; 30% of signups use Google
Passkeys / FIDO2 (roadmap 2025-2026)	Highest: device-bound cryptographic keys	TouchID / Face ID or security key; seamless	Planned for ShopFlow mobile app first

INFO Why Passkeys Matter for ShopFlow

ShopFlow processes payments — a compromised account means fraudulent orders. With passwords, attackers use credential stuffing (buying leaked username/password lists and trying them on shopflow.com). Cognito Advanced Security detects many of these attacks, but passkeys eliminate the attack vector entirely: there is no password to steal. The ShopFlow mobile app will implement passkeys via the WebAuthn API and Cognito's passkey support when it reaches GA.

17-30

End-to-End Authentication Flow — ShopFlow Login to API Call

Let's trace a complete ShopFlow customer login from clicking "Sign In" to receiving their order history. Every step maps to an AWS service from this chapter.

Step	Actor	AWS Service	What Happens
1. User clicks "Sign In"	React SPA	—	React renders login form; Amplify SDK ready to call Cognito
2. Submit email & password	React SPA → Cognito	Cognito User Pool	Amplify calls Cognito InitiateAuth API (SRP protocol — password never sent in plaintext)
3. Cognito verifies credentials	Cognito	Cognito User Pool + Lambda Trigger	Cognito checks password hash; Pre-TokenGeneration Lambda adds shopflowUserId to tokens
4. Tokens returned	Cognito → React	Cognito User Pool	id_token (60min), access_token (60min), refresh_token (30 days) returned to Amplify
5. React stores tokens	React SPA	—	Amplify stores tokens in memory (not localStorage); refresh token in httpOnly cookie
6. API request to /api/orders	React SPA → ALB	WAF + ALB	GET /api/orders HTTP header: Authorization: Bearer {access_token}; WAF checks request
7. WAF inspection	WAF	AWS WAF	WAF checks rate limits, OWASP rules, IP reputation. If OK: forwards to ALB
8. ALB validates JWT	ALB	ALB + Cognito	ALB calls Cognito JWKS endpoint to verify token signature. If expired: returns 401
9. Request forwarded	ALB → ECS	ALB + ECS	ALB adds X-Amzn-Oidc-Data header (decoded JWT) and forwards to ECS product endpoint
10. App processes request	ECS	Spring Boot + Aurora	Spring Boot reads userId from X-Amzn-Oidc-Data header; queries Aurora for that user's orders
11. Response returned	ECS → React	—	JSON order list returned; React renders order history page